

Findings from Closed-Form Locality Analysis

For fifty years, a kernel’s arithmetic cost has been a formula (matmul: $2n^3$) while its memory cost — the quantity that actually limits performance — has been an experiment: run it, or simulate it, once per machine, per input size, per program variant. The algebraic locality compiler closes that asymmetry for affine kernels (loops with linear subscripts: matmuls, convolutions, stencils, attention blocks): it derives, in about a minute per kernel, the complete distribution of *reuse distances* — for each memory access, how much other data was touched since the previous access to the same value — as exact polynomials in **all** loop bounds at once. Since an access hits a cache of capacity C exactly when its reuse distance is at most C , these polynomials are the kernel’s miss ratio *as a formula* in every problem parameter and the cache size simultaneously. Accuracy is settled in the paper ($\approx 1\%$ against cycle-level simulation across 41 kernels).

A capability, however, is only as interesting as what it finds. This report uses the tool as an instrument and reports findings — statements we did not know, could not have measured, and in one case would have gotten wrong by hand. The main study takes the attention family (softmax attention, linear/recurrent attention, chunked linear attention) with sequence length n and head dimension d as *separate* symbols; a second part reports what the same instrument found across the PolyBench suite. All claims pass the framework’s built-in conservation checks (Section 5); everything is derived, nothing is profiled.

Finding 1. Linear attention is context-length-free; its one cliff is in head dimension, exactly where the field operates

Linear (recurrent) attention maintains a $d \times d$ state S ; per token it updates S with $K_i \otimes V_i$ and emits $O_i = Q_i S$. Deriving its table (params n, d) and inspecting the reuse-distance formulas:

98.97% of all accesses have reuse distances whose formulas do not contain n . The only n -dependent distances are the whole-footprint (cross-invocation) reuses, carrying 0.11% of accesses (0.9% of mass is filtered by the analyzer; Section 5).

This is a for-all statement, not an extrapolation: the miss ratio of linear attention is the same at context 2k and context 65k — verified numerically to five digits ($mr = 0.03580$ at $n = 2048, 8192, 65536$; $d = 128, 32$ KB). No measurement campaign can establish independence; a formula free of n is the proof. Growing context costs linear attention arithmetic, but not one byte per token of extra traffic.

Its only cliff is in the head dimension. The largest n -free distance is the state-reuse knee — the $d \times d$ state plus one row set, e.g. 33.8 KB at $d = 64$ — and crossing it is violent:

d	mr @ 32 KB	per-token traffic @ 32 KB	@ 1 MB
48	0.0015	1.5 KB	1.5 KB
64	0.0359	64.3 KB	2 KB
128	0.0358	257 KB	4 KB
256	0.0358	1 MB	8 KB

A **43x per-token traffic jump** between $d = 48$ and $d = 64$ at 32 KB: once the state does not fit, it is re-streamed twice per token ($\approx 16d^2$ bytes), swamping the $32d$ -byte QKVO streaming that is all a resident state pays. The residency condition is $d^2 \cdot (\text{bytes/elt}) \leq C$, i.e. **$d \cdot (32 \text{ KB})$**

≈ 62 in fp64, 90 in fp32, 128 in bf16 — the precision-dependent boundary lands exactly on the $d = 64$ and $d = 128$ head dimensions modern models use: $d = 64$ heads are L1-resident in fp32/bf16 but not fp64; $d = 128$ only in bf16; $d = 256$ is L1-resident never, L2-resident always ($mr \leq 0.0006$ at 1 MB for all $d \leq 256$). Whether the field’s head sizes co-evolved with this boundary or merely collide with it, the boundary itself was not written down before; it falls out of the state-reuse formula.

In one sentence: softmax attention cliffs in context length, linear attention cliffs in head dimension — the two families’ cache behavior is organized along orthogonal parameters, and both cliff locations are closed-form.

Finding 2. Softmax attention’s cliff is $n^* = C/d$ — a 50x jump — and below it *all* traffic is the score matrix

Dense attention ($S = QK^T$, row softmax, $O = PV$; unfused, params n , d) per-token DRAM traffic from the tables, $d = 64$:

n	@ 32 KB	@ 1 MB	@ 32 MB
1024	1.04 MB	41.9 KB	0
2048	2.08 MB	2.08 MB	81.9 KB
8192	8.31 MB	8.31 MB	322 KB

The 1 MB column jumps **50x between $n = 1024$ and $n = 2048$** — and the formulas say exactly why and exactly where: the K/V panels ($n \cdot d$ elements) fall out of cache at

$$n^*(C, d) = C_{\text{elements}} / d \quad (= 2048 \text{ at } 1 \text{ MB}, d = 64; \\ 1024 \text{ at } d = 128 - \text{confirmed}),$$

after which both panels re-stream per token (16nd bytes). Below the cliff, the surviving traffic is $40n$ bytes per token — five streaming passes over the n -length score row (write S , two softmax reads, write P , read P) — which the numbers match to within 2% (41.9 KB vs $40 \cdot 1024 = 41$ KB). That is: **in the K/V-resident regime, essentially 100% of unfused attention’s DRAM traffic is the materialized score matrix — precisely the traffic FlashAttention-style fusion eliminates — and the regime’s extent, $n < C/d$, is now a formula rather than a rule of thumb.** Above n^* , fusion’s payoff changes character (K/V re-streaming joins the bill at 16nd bytes/token, 25x the score term at $d = 64$). The tool prices the fusion decision, with its validity boundary, per (n, d, C) .

Finding 3. Chunking linear attention has a computable free window — and naive working-set analysis mis-prices it

Practical linear-attention kernels process chunks of L tokens (dense $L \times L$ block within a chunk, state update between chunks) to regain matmul efficiency. The tables price the memory side of that trade ($n = 8192$):

per-token traffic	$d=64$ @32KB	$d=64$ @1MB	$d=128$ @1MB	$d=256$ @1MB
recurrent ($L=1$)	64.3 KB	2 KB	4 KB	8 KB
chunk 16	66.4 KB	2 KB	4 KB	8 KB
chunk 64	130 KB	2 KB	4 KB	20.4 KB
chunk 256	325 KB	5.12 KB	11.5 KB	22 KB

Three statements, none obvious in advance:

- **The recurrent form is the traffic floor everywhere.** Chunking buys arithmetic efficiency, never memory efficiency; the premium is the price of the $L \times L$ score block and its extra passes.
- **There is a free window and it is computable:** chunk 64 costs *zero* premium at 1 MB ($d \leq 128$) — its block working set stays resident — while chunk 256 pays 2.5x. The memory-optimal chunk is the largest L inside the window, and the window boundary comes out of the same tables.
- **Hand analysis gets chunk 256 wrong.** A working-set estimate says a 256-chunk at $d = 64$ (score block 512 KB + panels + state) fits 1 MB, predicting zero premium. The tables show 2.5x: the phase structure of the chunk (score block written in one nest, re-read two nests later while V streams through) evicts the block between phases. The symbolic analysis accounts for *when* data is touched, not just how much exists — and here that distinction changes the answer.

Finding 4. The dense/linear traffic ratio has a closed form

Combining the tables: per-token traffic ratio (dense / linear) at 1 MB follows $5n/4d$ while K/V stay resident — 21x at ($n = 1024, d = 64$), matching the formula — and after the dense cliff grows like $n/2$: 2,100x at 4k context, 34,000x at 64k. Everyone knows dense attention “moves more data”; the tables replace the sentiment with a two-regime law with constants. (Arithmetic comparisons are textbook — $O(n^2d)$ vs $O(nd^2)$; the memory law, with its cache-conditioned regime switch, is not.)

5. Why these numbers can be trusted (and when not)

Three gates, all mechanical:

- **Conservation.** Summed access fractions must equal the access count. All six attention kernels pass at 98-99.5% (the residual is the analyzer’s documented filtering of degenerate boundary regions) — including the *causal* variant (0.982-0.991), notable because triangular iteration spaces are where the tool’s distance approximation is weakest.
- **A sum rule.** Distances weighted by frequency must equal the data footprint exactly (a theorem of the model); on matmul it holds to 4e-5, and where it fails it *localizes* the filtered mass.
- **Exclusion, not absorption.** Applied to PolyBench, the same gates *caught* six kernels (time-stepped stencils) whose analyzer output violates conservation by up to 54%, and they are excluded from all suite claims; the triangular suite kernels (cholesky, lu, syrk) conserve but their distance values are under-resolved, so claims about them are withheld. An instrument that can convict its own implementation is doing something a profiler cannot: a wrong empirical curve looks exactly like a right one.

Honest scope notes: the attention DSLs model access patterns (softmax as streaming row passes; no data-dependent control — none exists in these kernels); the analyzer models one level of fully associative LRU at 64-byte lines of f64, so fp32/bf16 statements are unit rescalings; chunked variants omit intra-chunk masking.

6. The same instrument, pointed at a classical suite

Applied to 22 conserving PolyBench kernels (all parameters symbolic), the reading that matters is not “locality exists” but what the formulas settle:

- **Every staircase boundary in the suite sits at a simple rational power of the data footprint D** — $D^0, D^{1/4}, D^{1/3}, D^{1/2}, D^{2/3}, D$ — with constant or $1/n$ -decaying

miss plateaus between. Locality in this class is *quantized*; a kernel is summarized losslessly by a handful of (exponent, plateau) pairs. The folklore “ $\sqrt{2}$ rule” (double the data, $\sqrt{2}x$ the cache) is exactly the $D^{1/2}$ case, now with a per-kernel validity window — and a counterexample, trisolve, which has no intermediate boundary at all: pinned at 3.1% misses until the entire matrix fits, provably beyond help from tiling.

- **No single “data-movement” score can rank kernels — structurally.** The sum rule forces any distance-weighted scalar to be dominated by rows carrying vanishing access fractions, i.e. by behavior real caches never see. Concretely: gesummv scores worse than mvt yet misses 3.3x less at 32 KB; our earlier exponent-based study classified trmm as “no locality headroom” while the formulas show a 16x waste at 32 KB (and confirm trisolve’s genuine hopelessness — same asymptotic class, opposite truths).
- **Parallel decomposition is parameter substitution.** Slicing a loop across p workers substitutes that bound with n/p in already-derived formulas. For matmul this proves row-slicing moves identical total data at every p ($1.02e9$ lines, $p = 1..1008$) while column-slicing merges p 1-MB caches into one at exactly $p = n^2 \cdot 8B/C = 32$ (traffic collapses 61x) — and exposes a line-granularity floor (a sliced matrix still spans $\geq n$ cache lines = 126 KB, so no worker count ever reaches L1 residency). The general rule is checkable per kernel from the tables: slicing helps only if the sliced loop indexes the array carrying the dominant reuse.

7. What to take away

The instrument’s outputs are not predictions to be checked against a run; they are *laws with validity regions*: linear attention’s context-freeness (a property no finite set of measurements can establish), residency boundaries that land on the field’s chosen head dimensions, a 50x attention cliff at $n = C/d$, fusion and chunking priced with the regime in which the price applies, quantized locality exponents across a numerical suite, impossibility results for scalar locality scores, and parallel-slicing laws obtained by substitution into formulas derived once. Where the instrument’s own gates fail, it says so instead of producing plausible noise.

That is the answer to “we made memory behavior symbolic — so what”: the same jump arithmetic complexity made long ago. When cost became a formula, questions stopped being experiments. This work does it for the half of performance that experiments were still required for.

Appendix: provenance and reproduction

Analyzer: AutoLALA dmd-cli, infinite repeat, scale approximation in Barvinok, block size 8 (64-byte lines of f64). Attention kernels in `dsl/att_*.dsl` (dense unfused with two softmax row passes; causal triangle; recurrent linear; chunked linear at $L = 16/64/256$ with symbolic chunk count), written for this study; PolyBench DSLs in `dsl/sym_*.dsl`; matmul family (matmul3*) as validation witnesses — the pipeline reproduces the paper’s Table 1 exactly and Table 6 structurally (tables/anchors.md).

Pipeline: `run_suite.py` (analyzer runs) $\rightarrow$ `regimes.py` (exact symbolic extraction; Fraction arithmetic; region-domain gating) $\rightarrow$ `derived.py`, `parallel_study.py`, `anchor_checks.py`, `attention_study.py` $\rightarrow$ tables/ (attention.md, machine_map.md, tiling.md, parallel.md, signatures.md, dmd_inversion.md, suite_regimes.md, anchors.md). Conservation excluded heat-3d (1.54), jacobi-1d (1.33), jacobi-2d (1.27), seidel-2d (0.93), ftd (0.96), imperfect (0.97). Raw analyzer JSON (data/, ~30 MB) is gitignored and regenerable.

```
python3 run_suite.py && python3 regimes.py && python3 derived.py
python3 parallel_study.py && python3 anchor_checks.py
python3 attention_study.py
```

```
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \  
-V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono" \  
-V geometry:margin=1in
```